

README.md – Python + uv + Jupyter (Enterprise Setup & Daily Workflow)

Python + uv + Jupyter (Enterprise-Grade Setup)

This repository uses a **corporate-safe Python workflow** suitable for managed enterprise devices (no admin access, no system PATH changes).

✓ Assumptions / Constraints

- Python 3.14.x is IT-installed (e.g. `C:\Program Files\Python314\`)
 - No admin privileges
 - No system-level PATH modification
 - All dependencies live **inside a project virtual environment**
-

📁 Repository Expectations

Plain Text ▾

```
1 <repo-root>/
2 |— .venv/           # local virtual environment (NOT committed)
3 |— notebooks/      # Jupyter notebooks (learning & experiments)
4 |— src/             # reusable Python modules
5 |— requirements.txt # locked dependencies
6 |— .env.example     # environment variable template
7 |— .gitignore
8 |— README.md
9
```

🔧 One-Time Setup (After Cloning the Repo)

Perform these steps **once per machine**.

1 Clone the Repository

Shell ▾

```
1 git clone <repo-url>
2 cd <repo-name>
3
```

✓ Check:

Shell ▾

```
1 git status
2
```

2 Create Virtual Environment (Inside Repo)

Shell ▾

```
1 py -m venv .venv
2
```

3 Activate Virtual Environment (Git Bash)

Shell ▾

```
1 source .venv/Scripts/activate
2
```

✓ Check:

```
1 which python
2 python --version
3
```

Shell ▾

Expected:

- Python path → `.venv/Scripts/python.exe`
- Version → `3.14.x`

4 Bootstrap pip (Enterprise-Safe)

Some corporate Windows builds create venv **without** pip.

```
1 python -m ensurepip --upgrade
2 python -m pip install --upgrade pip
3
```

Shell ▾

✓ Check:

```
1 python -m pip --version
2
```

Shell ▾

5 Install uv (Inside venv)

```
1 python -m pip install uv
2
```

Shell ▾

✓ Check:

```
1 python -m uv --version
2
```

Shell ▾

i Best practice: always run uv as `python -m uv` (PATH-independent).

6 Install JupyterLab + Kernel

```
1 python -m uv pip install jupyterlab ipykernel
2
```

Shell ▾

Register kernel (one-time):

```
1 python -m ipykernel install --user \  
2 --name genai-uv \  
3 --display-name "Python (genai-uv)"  
4
```

Shell ▾

7 Lock Dependencies

```
1 python -m uv pip freeze > requirements.txt
2
```

Shell ▾

✓ Commit `requirements.txt`.

8 Start JupyterLab

1 `python -m jupyter lab`
2

Shell ▾

✓ Open browser → `http://localhost:8888` ✓ Select kernel → **Python (genai-uv)**



Day-Start Checklist (EVERY WORKING DAY)

This is the **most important section**. Follow this daily.

- 1 ☐ Open Git Bash
- 2 ☐ cd into repo root
- 3 ☐ Activate virtual environment
- 4 ☐ Start JupyterLab
- 5 ☐ Verify correct kernel
- 6

Plain Text ▾

Commands

1 `cd <repo-name>`
2 `source .venv/Scripts/activate`
3 `python --version`
4 `python -m jupyter lab`
5

Shell ▾

✓ Confirm inside Jupyter:

- Kernel → Python (genai-uv)

Day-End Checklist

Plain Text ▾

- 1 ☐ Stop JupyterLab
- 2 ☐ Deactivate virtual environment
- 3 ☐ Commit daily work
- 4

Commands

Shell ▾

```
1 # Stop Jupyter
2 Ctrl + C
3
4 # Exit venv
5 deactivate
6
7 # Save progress
8 git status
9 git add .
10 git commit -m "Day X: summary of work"
11 git push
12
```

Re-Opening After System Restart / Next Day

✅ Do NOT reinstall anything ✅ Do NOT recreate venv

Only run:

Shell ▾

```
1 cd <repo-name>
2 source .venv/Scripts/activate
3 python -m jupyter lab
4
```

Common Pitfalls & Fallbacks

No module named pip

Shell ▾

```
1 python -m ensurepip --upgrade
2
```

uv not found

Always run:

Shell ▾

```
1 python -m uv <command>
2
```

Jupyter kernel not visible

Re-register kernel:

Shell ▾

```
1 python -m ipykernel install --user \
2   --name genai-uv \
3   --display-name "Python (genai-uv)"
4
```

Reset Environment (Last Resort)

Shell ▾

```
1 deactivate
2 rm -rf .venv
3 py -m venv .venv
4 source .venv/Scripts/activate
5 python -m ensurepip --upgrade
6 python -m pip install uv jupyterlab ipykernel
7
```

Mental Model (Remember This)

One repo → one venv → one Jupyter kernel

- Git = history
- venv = dependencies
- Jupyter = experimentation

This mirrors **real GenAI delivery projects** and is safe for enterprise environments.